

WebSocket with Spring Framework



Content :

1. Introduction

WebSocket and STOMP Protocols

2. Message Configuration

Message Broker Configuration.

3. WebSocket Session Manager

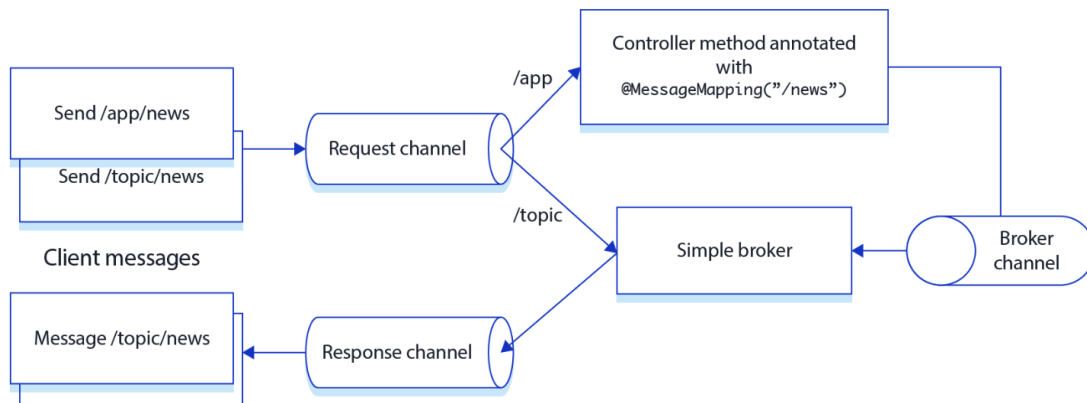
Session Active Management.

4. User Message Management

Message sending.

Introduction :

WebSockets is a bidirectional, full-duplex, persistent connection between a web browser and a server. Once a WebSocket connection is established, the connection stays open until the client or server decides to close this connection.



WebSocket and STOMP Protocols :

The WebSocket protocol allows you to implement bidirectional communication between applications. It is important to know that HTTP is used only for the initial handshake. After it happens, the HTTP connection is upgraded to a newly opened TCP/IP connection that is used by a WebSocket.

The WebSocket protocol is a rather low-level protocol. It defines how a stream of bytes is transformed into frames. A frame may contain a text or a binary message. Because the message itself does not provide any additional information on how to route or process it, It is difficult to implement more complex applications without writing additional code. Fortunately, the WebSocket specification allows the use of sub-protocols that operate on a higher, application level. One of them, supported by the Spring Framework, is STOMP.

STOMP is a simple text-based messaging protocol that was initially created for scripting languages such as Ruby, Python, and Perl to connect to enterprise message brokers. STOMP, clients and brokers developed in different languages can send and receive messages to and from each other. The WebSocket protocol is sometimes called TCP for Web. Analogically, STOMP is called HTTP for Web. It defines a handful of frame types that are mapped onto WebSockets frames, e.g., `CONNECT`, `SUBSCRIBE`, `UNSUBSCRIBE`, `ACK`, or `SEND`. On one hand, these commands are very handy to manage communication while, on the other, they allow us to implement solutions with more sophisticated features like message acknowledgment.

Message Configuration :

WebSocket in java the setup with **spring-boot-starter-websocket** dependency can be implemented starting with Configuration.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

Message Broker Configuration :

1. Creates the in-memory **message broker** with one or more destinations for sending and receiving messages. In the example above, there are two destination prefixes defined: `topic` and `queue`. They follow the convention that destinations for messages to be carried on to all subscribed clients via the pub-sub model should be prefixed with `topic`. On the other hand, destinations for private messages are typically prefixed by `queue`.

```

Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void registerStompEndpoints (StompEndpointRegistry
registry) {
        registry.addEndpoint ("/mywebsockets")
            .setAllowedOrigins ("mydomain.com").withSockJS ();
    }

    @Override
    public void configureMessageBroker (MessageBrokerRegistry config){
        config.enableSimpleBroker ("/topic/", "/queue/");
        config.setApplicationDestinationPrefixes ("/app");
    }
}

```

2. Defines the prefix app that is used to filter destinations handled by methods annotated with `@MessageMapping` which you will implement in a controller. The controller, after processing the message, will send it to the broker.

WebSocket Session Manager :

The sessions are managed by the connection event listening.

Session Active Management :

- When the Connection establishes the methods with the annotation of `EventListener` receive `SessionConnectEvent` in its listener.
- When the Connection is disconnected while the manual disconnection from frontend or tab was closed with the annotation of `EventListener` receive `SessionDisconnectEvent` in its listener.

```

@EventListener
public void handleWebSocketConnectListener(SessionConnectEvent event) {
    StompHeaderAccessor headers = StompHeaderAccessor.wrap(event.getMessage());
    String sessionId = headers.getSessionId();
    log.info(sessionId);
}

@EventListener
public void handleWebSocketDisconnectListener(SessionDisconnectEvent event) {
    StompHeaderAccessor headers = StompHeaderAccessor.wrap(event.getMessage());
    String sessionId = headers.getSessionId();
    log.info(sessionId);
}

```

User Message Management :

Message sending :

The sending message from the application can be accessed through MessageMapping endpoints for various kinds of messaging service.

```

@MessageMapping("/private")
public Message connect(@Payload Message message) {
    messageService.sendPrivateMessage(message);
    return message;
}

```

Private message will be received for a specific user sending it with a private endpoint with a user id and Public message will be received for all users.

```

@Override
public void sendPublicMessage(Message message) {
    simpMessagingTemplate.convertAndSend("/public/message", message);
}

@Override
public void sendPrivateMessage(Message message) {
    simpMessagingTemplate.convertAndSendToUser(message.getReceiver().toString(), "/message", message);
}

```